

been given by Lyness in [4]; for related references, see [5]. To our knowledge, the cubic-order formulas derived here have not previously appeared in the literature.

Calculating Fourier transforms when the range of integration is $(-\infty, \infty)$ can be tricky. If the function falls off reasonably quickly at infinity, you can split the integral at a large enough value of t . For example, the integration to $+\infty$ can be written

$$\begin{aligned}\int_a^\infty e^{i\omega t} h(t) dt &= \int_a^b e^{i\omega t} h(t) dt + \int_b^\infty e^{i\omega t} h(t) dt \\ &= \int_a^b e^{i\omega t} h(t) dt - \frac{h(b)e^{i\omega b}}{i\omega} + \frac{h'(b)e^{i\omega b}}{(i\omega)^2} - \dots\end{aligned}\quad (13.9.17)$$

The splitting point b must be chosen large enough that the remaining integral over (b, ∞) is small. Successive terms in its asymptotic expansion are found by integrating by parts. The integral over (a, b) can be done using `dftint`. You keep as many terms in the asymptotic expansion as you can easily compute. See [6] for some examples of this idea. More powerful methods, which work well for long-tailed functions but which do not use the FFT, are described in [7-9].

CITED REFERENCES AND FURTHER READING:

- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §2.3.4.[1]
- Narasimhan, M.S. and Karthikeyan, M. 1984, "Evaluation of Fourier Integrals Using a FFT with Improved Accuracy and Its Applications," *IEEE Transactions on Antennas and Propagation*, vol. 32, pp. 404–408.[2]
- Filon, L.N.G. 1928, "On a Quadrature Formula for Trigonometric Integrals," *Proceedings of the Royal Society of Edinburgh*, vol. 49, pp. 38–47.[3]
- Giunta, G. and Murli, A. 1987, "A Package for Computing Trigonometric Fourier Coefficients Based on Lyness's Algorithm," *ACM Transactions on Mathematical Software*, vol. 13, pp. 97–107.[4]
- Lyness, J.N. 1987, in *Numerical Integration*, P. Keast and G. Fairweather, eds. (Dordrecht: Reidel).[5]
- Pantis, G. 1975, "The Evaluation of Integrals with Oscillatory Integrands," *Journal of Computational Physics*, vol. 17, pp. 229–233.[6]
- Blakemore, M., Evans, G.A., and Hyslop, J. 1976, "Comparison of Some Methods for Evaluating Infinite Range Oscillatory Integrals," *Journal of Computational Physics*, vol. 22, pp. 352–376.[7]
- Lyness, J.N., and Kaper, T.J. 1987, "Calculating Fourier Transforms of Long Tailed Functions," *SIAM Journal on Scientific and Statistical Computing*, vol. 8, pp. 1005–1011.[8]
- Thakkar, A.J., and Smith, V.H. 1975, "A Strategy for the Numerical Evaluation of Fourier Sine and Cosine Transforms to Controlled Accuracy," *Computer Physics Communications*, vol. 10, pp. 73–79.[9]

13.10 Wavelet Transforms

Like the fast Fourier transform (FFT), the discrete wavelet transform (DWT) is a fast, linear operation that operates on a data vector whose length is an integer power of 2, transforming it into a numerically different vector of the same length. Also like the FFT, the wavelet transform is invertible and in fact orthogonal — the inverse transform, when viewed as a big matrix, is simply the transpose of the transform.

Both FFT and DWT, therefore, can be viewed as a rotation in function space, from the input space (or time) domain, where the basis functions are the unit vectors $\mathbf{e}_i$, or Dirac delta functions in the continuum limit, to a different domain. For the FFT, this new domain has basis functions that are the familiar sines and cosines. In the wavelet domain, the basis functions are somewhat more complicated and have the fanciful names “mother functions” and “wavelets.”

Of course there are an infinity of possible bases for function space, almost all of them uninteresting! What makes the wavelet basis interesting is that, *unlike* sines and cosines, individual wavelet functions are quite localized in space; simultaneously, *like* sines and cosines, individual wavelet functions are quite localized in frequency or (more precisely) characteristic scale. As we will see below, the particular kind of dual localization achieved by wavelets renders large classes of functions and operators sparse, or sparse to some high accuracy, when transformed into the wavelet domain. Analogously with the Fourier domain, where a class of computations, like convolutions, becomes computationally fast, there is a large class of computations — those that can take advantage of sparsity — that becomes computationally fast in the wavelet domain [1].

Unlike sines and cosines, which define a unique Fourier transform, there is not one single unique set of wavelets; in fact, there are infinitely many possible sets. Roughly, the different sets of wavelets make different trade-offs between how compactly they are localized in space, how smooth they are, and whether they have any special boundary conditions. (There are further fine distinctions.)

13.10.1 Daubechies Wavelet Filter Coefficients

A particular set of wavelets is specified by a particular set of numbers, called *wavelet filter coefficients*. Here, we will largely restrict ourselves to wavelet filters in a class discovered by Daubechies [2]. This class includes members ranging from highly localized to highly smooth. The simplest (and most localized) member, often called *DAUB4*, has only four coefficients, $c_0, \dots, c_3$. For the moment we specialize to this case for ease of notation.

Consider the following transformation matrix acting on a column vector of data to its right:

$$\begin{bmatrix} c_0 & c_1 & c_2 & c_3 & & & & & \\ c_3 & -c_2 & c_1 & -c_0 & & & & & \\ & & c_0 & c_1 & c_2 & c_3 & & & \\ & & c_3 & -c_2 & c_1 & -c_0 & & & \\ \vdots & \vdots & & & & & \ddots & & \\ & & & & & & & c_0 & c_1 & c_2 & c_3 \\ & & & & & & & c_3 & -c_2 & c_1 & -c_0 \\ c_2 & c_3 & & & & & & & & c_0 & c_1 \\ c_1 & -c_0 & & & & & & & & c_3 & -c_2 \end{bmatrix} \quad (13.10.1)$$

Here blank entries signify zeroes. Note the structure of this matrix. The first row convolves four consecutive data points with the filter coefficients $c_0, \dots, c_3$; likewise, the third, fifth, and other odd rows. If the even rows followed this pattern, offset by one, then the matrix would be a circulant, that is, an ordinary convolution that could be done by FFT methods. (Note how the last two rows wrap around like convolutions

with periodic boundary conditions.) Instead of convolving with $c_0, \dots, c_3$, however, the even rows perform a different convolution, with coefficients $c_3, -c_2, c_1, -c_0$. The action of the matrix, overall, is thus to perform two related convolutions, then to decimate each of them by half (throw away half the values), and interleave the remaining halves.

It is useful to think of the filter $c_0, \dots, c_3$ as being a smoothing filter, call it H , something like a moving average of four points. Then, because of the minus signs, the filter $c_3, -c_2, c_1, -c_0$, call it G , is *not* a smoothing filter. (In signal processing contexts, H and G are called *quadrature mirror filters* [3].) In fact, the c 's are chosen so as to make G yield, insofar as possible, a *zero* response to a sufficiently smooth data vector. This is done by requiring the sequence $c_3, -c_2, c_1, -c_0$ to have a certain number of vanishing moments. When this is the case for p moments (starting with the zeroth), a set of wavelets is said to satisfy an “approximation condition of order p .” This results in the output of H , decimated by half, accurately representing the data’s “smooth” information. The output of G , also decimated, is referred to as the data’s “detail” information [4].

For such a characterization to be useful, it must be possible to reconstruct the original data vector of length N from its $N/2$ smooth or s-components and its $N/2$ detail or d-components. That is effected by requiring the matrix (13.10.1) to be orthogonal, so that its inverse is just the transposed matrix

$$\begin{bmatrix} c_0 & c_3 & & \cdots & & c_2 & c_1 \\ c_1 & -c_2 & & \cdots & & c_3 & -c_0 \\ c_2 & c_1 & c_0 & c_3 & & & \\ c_3 & -c_0 & c_1 & -c_2 & & & \\ & & & \ddots & & & \\ & & & & c_2 & c_1 & c_0 & c_3 \\ & & & & c_3 & -c_0 & c_1 & -c_2 \\ & & & & & & c_2 & c_1 & c_0 & c_3 \\ & & & & & & c_3 & -c_0 & c_1 & -c_2 \end{bmatrix} \quad (13.10.2)$$

One sees immediately that matrix (13.10.2) is inverse to matrix (13.10.1) if and only if these two equations hold,

$$\begin{aligned} c_0^2 + c_1^2 + c_2^2 + c_3^2 &= 1 \\ c_2c_0 + c_3c_1 &= 0 \end{aligned} \quad (13.10.3)$$

If additionally we require the approximation condition of order $p = 2$, then two additional relations are required,

$$\begin{aligned} c_3 - c_2 + c_1 - c_0 &= 0 \\ 0c_3 - 1c_2 + 2c_1 - 3c_0 &= 0 \end{aligned} \quad (13.10.4)$$

Equations (13.10.3) and (13.10.4) are four equations for the four unknowns $c_0, \dots, c_3$, first recognized and solved by Daubechies. The unique solution (up to a left-right reversal) is

$$\begin{aligned} c_0 &= (1 + \sqrt{3})/4\sqrt{2} & c_1 &= (3 + \sqrt{3})/4\sqrt{2} \\ c_2 &= (3 - \sqrt{3})/4\sqrt{2} & c_3 &= (1 - \sqrt{3})/4\sqrt{2} \end{aligned} \quad (13.10.5)$$

In fact, DAUB4 is only the most compact of a sequence of wavelet sets: If we had six coefficients instead of four, there would be three orthogonality requirements in equation (13.10.3) (with offsets of zero, two, and four), and we could require the vanishing of $p = 3$ moments in equation (13.10.4). In this case, DAUB6, the solution coefficients can also be expressed in closed form,

$$\begin{aligned} c_0 &= (1 + \sqrt{10} + \sqrt{5 + 2\sqrt{10}})/16\sqrt{2} & c_1 &= (5 + \sqrt{10} + 3\sqrt{5 + 2\sqrt{10}})/16\sqrt{2} \\ c_2 &= (10 - 2\sqrt{10} + 2\sqrt{5 + 2\sqrt{10}})/16\sqrt{2} & c_3 &= (10 - 2\sqrt{10} - 2\sqrt{5 + 2\sqrt{10}})/16\sqrt{2} \\ c_4 &= (5 + \sqrt{10} - 3\sqrt{5 + 2\sqrt{10}})/16\sqrt{2} & c_5 &= (1 + \sqrt{10} - \sqrt{5 + 2\sqrt{10}})/16\sqrt{2} \end{aligned} \quad (13.10.6)$$

For higher p , the coefficients are available only numerically, e.g., tabulated in [5] or [6]. (We use some of these below.) The number of coefficients increases by two each time p is increased by one.

13.10.2 Discrete Wavelet Transform

We have not yet defined the discrete wavelet transform (DWT), but we are almost there: The DWT consists of applying a wavelet coefficient matrix like (13.10.1) *hierarchically*, first to the full data vector of length N , then to the “smooth” vector of length $N/2$, then to the “smooth-smooth” vector of length $N/4$, and so on until only a trivial number of “smooth-...-smooth” components (usually 2 or 4) remain. The procedure is sometimes called a *pyramidal algorithm* [4], for obvious reasons. The output of the DWT consists of these remaining components and all the “detail” components that were accumulated along the way. A diagram should make the procedure clear:

$$\begin{array}{ccccccc} \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \\ y_7 \\ y_8 \\ y_9 \\ y_{10} \\ y_{11} \\ y_{12} \\ y_{13} \\ y_{14} \\ y_{15} \end{bmatrix} & \xrightarrow{13.10.1} & \begin{bmatrix} s_0 \\ d_0 \\ s_1 \\ d_1 \\ s_2 \\ d_2 \\ s_3 \\ d_3 \\ s_4 \\ d_4 \\ s_5 \\ d_5 \\ s_6 \\ d_6 \\ s_7 \\ d_7 \end{bmatrix} & \xrightarrow{\text{permute}} & \begin{bmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \\ s_6 \\ s_7 \\ d_0 \\ d_1 \\ d_2 \\ d_3 \\ d_4 \\ d_5 \\ d_6 \\ d_7 \end{bmatrix} & \xrightarrow{13.10.1} & \begin{bmatrix} S_0 \\ D_0 \\ S_1 \\ D_1 \\ S_2 \\ D_2 \\ S_3 \\ D_3 \\ d_0 \\ d_1 \\ d_2 \\ d_3 \\ d_4 \\ d_5 \\ d_6 \\ d_7 \end{bmatrix} \\ & & & & \xrightarrow{\text{permute}} & & \begin{bmatrix} S_0 \\ S_1 \\ S_2 \\ S_3 \\ D_0 \\ D_1 \\ D_2 \\ D_3 \\ d_0 \\ d_1 \\ d_2 \\ d_3 \\ d_4 \\ d_5 \\ d_6 \\ d_7 \end{bmatrix} \\ & & & & & \xrightarrow{\text{etc.}} & \begin{bmatrix} \mathcal{S}_0 \\ \mathcal{S}_1 \\ \mathcal{D}_0 \\ \mathcal{D}_1 \\ D_0 \\ D_1 \\ D_2 \\ D_3 \\ d_0 \\ d_1 \\ d_2 \\ d_3 \\ d_4 \\ d_5 \\ d_6 \\ d_7 \end{bmatrix} \end{array} \quad (13.10.7)$$

If the length of the data vector were a higher power of 2, there would be more stages of applying (13.10.1) (or any other wavelet coefficients) and permuting. The endpoint will always be a vector with two $\mathcal{S}$ ’s and a hierarchy of $\mathcal{D}$ ’s, D ’s, d ’s,

etc. Notice that once d 's are generated, they simply propagate through to all subsequent stages.

A value d_i of any level is termed a “wavelet coefficient” of the original data vector; the final values $\mathcal{S}_0, \mathcal{S}_1$ should strictly be called “mother-function coefficients,” although the term “wavelet coefficients” is often used loosely for both d 's and final $\mathcal{S}$'s. Since the full procedure is a composition of orthogonal linear operations, the whole DWT is itself an orthogonal linear operator.

To invert the DWT, one simply reverses the procedure, starting with the smallest level of the hierarchy and working (in equation 13.10.7) from right to left. The inverse matrix (13.10.2) is of course used instead of the matrix (13.10.1).

As already noted, the matrices (13.10.1) and (13.10.2) embody periodic (“wrap-around”) boundary conditions on the data vector. One normally accepts this as a minor inconvenience: The last few wavelet coefficients at each level of the hierarchy are affected by data from both ends of the data vector. By circularly shifting the matrix (13.10.1) $N/2$ columns to the left, one can symmetrize the wraparound; but this does not eliminate it. It is in fact possible to eliminate the wraparound completely by altering the coefficients in the first and last few rows of (13.10.1), giving an orthogonal matrix that is purely band-diagonal. This variant can be useful when, e.g., the data vary by many orders of magnitude from one end of the data vector to the other. We discuss it in §13.10.5, below.

Here is a DWT routine, `wt1`, that performs the pyramidal algorithm (or its inverse if `isign` is negative) on some data vector `a[0..n-1]`. Successive applications of the wavelet filter, and accompanying permutations, are performed by the object `wlet`, of class `Wavelet`, to be described below. The routine `wt1` also provides for the possibility of preconditioning and postconditioning steps, which we won't need until a later subsection.

```
void wt1(VecDoub_IO &a, const Int isign, Wavelet &wlet)
```

[wavelet.h](#)

One-dimensional discrete wavelet transform. This routine implements the pyramid algorithm, replacing `a[0..n-1]` by its wavelet transform (for `isign=1`), or performing the inverse operation (for `isign=-1`). Note that `n` MUST be an integer power of 2. The object `wlet`, of type `Wavelet`, is the underlying wavelet filter. Examples of `Wavelet` types are `Daub4`, `Daubs`, and `Daub4i`.

```
{
    Int nn, n=a.size();
    if (n < 4) return;
    if (isign >= 0) {
        wlet.condition(a,n,1);
        for (nn=n;nn>=4;nn>=1) wlet.filt(a,nn,isign);
        Start at largest hierarchy, and work toward smallest.
    } else {
        for (nn=4;nn<=n;nn<=1) wlet.filt(a,nn,isign);
        Start at smallest hierarchy, and work toward largest.
        wlet.condition(a,n,-1);
    }
}
```

The `Wavelet` class is an “abstract base class,” meaning that it is really only a promise that specific wavelets that derive from it will contain a method called `filt`, the actual wavelet filter. `Wavelet` also provides a default, null, pre- and post-conditioning method. The class `Daub4` is derived from `Wavelet` and is intended for use with `wt1`. Its `filt` method implements the matrices (13.10.1) and (13.10.2), along with the permutation shown in (13.10.7).

```

wavelet.h struct Wavelet {
    virtual void filt(VecDoub_IO &a, const Int n, const Int isign) = 0;
    virtual void condition(VecDoub_IO &a, const Int n, const Int isign) {}
};

struct Daub4 : Wavelet {
    void filt(VecDoub_IO &a, const Int n, const Int isign) {
        Applies the Daubechies 4-coefficient wavelet filter to data vector a[0..n-1] (for isign=1)
        or applies its transpose (for isign=-1). Used hierarchically by routines wt1 and wtn.
        const Doub C0=0.4829629131445341, C1=0.8365163037378077,
        C2=0.2241438680420134, C3=-0.1294095225512603;
        Int nh,i,j;
        if (n < 4) return;
        VecDoub wksp(n);
        nh = n >> 1;
        if (isign >= 0) {
            Apply filter.
            for (i=0,j=0;j<n-3;j+=2,i++) {
                wksp[i] = C0*a[j]+C1*a[j+1]+C2*a[j+2]+C3*a[j+3];
                wksp[i+nh] = C3*a[j]-C2*a[j+1]+C1*a[j+2]-C0*a[j+3];
            }
            wksp[i] = C0*a[n-2]+C1*a[n-1]+C2*a[0]+C3*a[1];
            wksp[i+nh] = C3*a[n-2]-C2*a[n-1]+C1*a[0]-C0*a[1];
        } else {
            Apply transpose filter.
            wksp[0] = C2*a[nh-1]+C1*a[n-1]+C0*a[0]+C3*a[nh];
            wksp[1] = C3*a[nh-1]-C0*a[n-1]+C1*a[0]-C2*a[nh];
            for (i=0,j=2;i<nh-1;i++) {
                wksp[j++] = C2*a[i]+C1*a[i+nh]+C0*a[i+1]+C3*a[i+nh+1];
                wksp[j++] = C3*a[i]-C0*a[i+nh]+C1*a[i+1]-C2*a[i+nh+1];
            }
        }
        for (i=0;i<n;i++) a[i]=wksp[i];
    }
};

```

For larger sets of wavelet coefficients, the wraparound of the last rows or columns is a programming inconvenience. An efficient implementation would handle the wraparounds as special cases, outside of the main loop. For now, we will content ourselves with a more general scheme involving some extra arithmetic at run-time.

The following class, Daubs, takes an integer argument n in its constructor and creates a wavelet object with the filter DAUB n . Slightly better than “Hobson’s choice,” you can choose $n = 4, 12$, or 20 . For other values of n you will need to add additional coefficient tables (e.g., from [6]).

```

wavelet.h struct Daubs : Wavelet {
    Structure for initializing and using the DAUB $n$  wavelet filter for any  $n$  whose coefficients are
    provided (here  $n = 4, 12, 20$ ).
    Int ncof,ioff,joff;
    VecDoub cc,cr;
    static Doub c4[4],c12[12],c20[20];
    Daubs(Int n) : ncof(n), cc(n), cr(n) {
        Int i;
        ioff = joff = -(n >> 1);
        // ioff = -2; joff = -n + 2; Alternative centering. (Used by Daub4, above.)
        if (n == 4) for (i=0; i<n; i++) cc[i] = c4[i];
        else if (n == 12) for (i=0; i<n; i++) cc[i] = c12[i];
        else if (n == 20) for (i=0; i<n; i++) cc[i] = c20[i];
        else throw("n not yet implemented in Daubs");
        Doub sig = -1.0;
        for (i=0; i<n; i++) {
            cr[n-1-i]=sig*cc[i];
            sig = -sig;
        }
    }
};

```

```

    }
}
void filt(VecDoub_IO &a, const Int n, const Int isign); See below.
};

Doub Daubs::c4[4]=
{0.4829629131445341,0.8365163037378079,
 0.2241438680420134,-0.1294095225512604};
Doub Daubs::c12[12]=
{0.111540743350, 0.494623890398, 0.751133908021,
 0.315250351709,-0.226264693965,-0.129766867567,
 0.097501605587, 0.027522865530,-0.031582039318,
 0.000553842201, 0.004777257511,-0.001077301085};
Doub Daubs::c20[20]=
{0.026670057901, 0.188176800078, 0.527201188932,
 0.688459039454, 0.281172343661,-0.249846424327,
 -0.195946274377, 0.127369340336, 0.093057364604,
 -0.071394147166,-0.029457536822, 0.033212674059,
 0.003606553567,-0.010733175483, 0.001395351747,
 0.001992405295,-0.000685856695,-0.000116466855,
 0.000093588670,-0.000013264203};

```

There is some arbitrariness in how the wavelets at each hierarchical stage are centered over the data they act on. Daubs implements one popular choice, with another shown in commented code. Consult the literature if this matters to you (it rarely does).

The implementation of `Daubs::filt()` is straightforward:

```

void Daubs::filt(VecDoub_IO &a, const Int n, const Int isign) {
Applies the previously initialized Daub $n$  wavelet filter to data vector a[0..n-1] (for isign = 1)
or applies its transpose (for isign = -1). Used hierarchically by routines wt1 and wtn.
    Doub ai,ai1;
    Int i,ii,j,jf,jr,k,n1,ni,nj,nh,nmod;
    if (n < 4) return;
    VecDoub wksp(n);
    nmod = ncof*n;
    n1 = n-1;
    nh = n >> 1;
    for (j=0;j<n;j++) wksp[j]=0.0;
    if (isign >= 0) {
        for (ii=0,i=0;i<n;i+=2,ii++) {
            ni = i+1+nmod+ioff;
            nj = i+1+nmod+joff;
            for (k=0;k<ncof;k++) {
                jf = n1 & (ni+k+1);
                jr = n1 & (nj+k+1);
                wksp[ii] += cc[k]*a[jf];
                wksp[ii+nh] += cr[k]*a[jr];
            }
        }
    }
    } else {
        for (ii=0,i=0;i<n;i+=2,ii++) {
            ai = a[ii];
            ai1 = a[ii+nh];
            ni = i+1+nmod+ioff;
            nj = i+1+nmod+joff;
            for (k=0;k<ncof;k++) {
                jf = n1 & (ni+k+1);
                jr = n1 & (nj+k+1);
                wksp[jf] += cc[k]*ai;
                wksp[jr] += cr[k]*ai1;
            }
        }
    }
}

```

wavelet.h

A positive constant equal to zero mod n .
Mask of all bits, since n a power of 2.

Apply filter.

Pointer to be incremented and wrapped around.

We use “bitwise and” to wrap around the pointers.

Apply transpose filter.

See comments above.

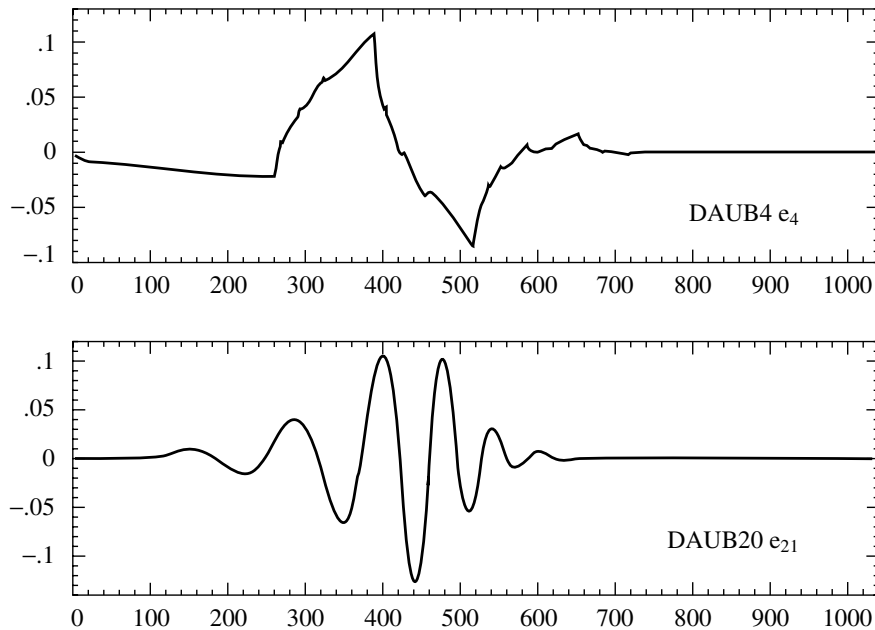


Figure 13.10.1. Wavelet functions, that is, single basis functions from the wavelet families DAUB4 and DAUB20. A complete, orthonormal wavelet basis consists of scalings and translations of either one of these functions. DAUB4 has an infinite number of cusps; DAUB20 would show similar behavior in a higher derivative.

```

    }
  }
  for (j=0;j<n;j++) a[j] = wksp[j];      Copy the results back from workspace.
}

```

13.10.3 What Do Wavelets Look Like?

We are now in a position to actually see some wavelets. To do so, we simply run unit vectors through any of the above discrete wavelet transforms, with `isign` negative so that the inverse transform is performed. Figure 13.10.1 shows the DAUB4 wavelet that is the inverse DWT of a unit vector in component 4 of a vector of length 1024, and also the DAUB20 wavelet that is the inverse of component 21. (One needs to go to a later hierarchical level for DAUB20 to avoid a wavelet with a wrapped-around tail.) Other unit vectors would give wavelets with the same shapes but different positions and scales.

One sees that both DAUB4 and DAUB20 have wavelets that are continuous. DAUB20 wavelets also have higher continuous derivatives. DAUB4 has the peculiar property that its derivative exists only *almost* everywhere. Examples of where it fails to exist are the points $p/2^n$, where p and n are integers; at such points, DAUB4 is left differentiable, but not right differentiable! This kind of discontinuity — at least in some derivative — is a necessary feature of wavelets with compact support, like the Daubechies series. For every increase in the number of wavelet coefficients by two, the Daubechies wavelets gain about *half* a derivative of continuity. (But not exactly half; the actual orders of regularity are irrational numbers!)

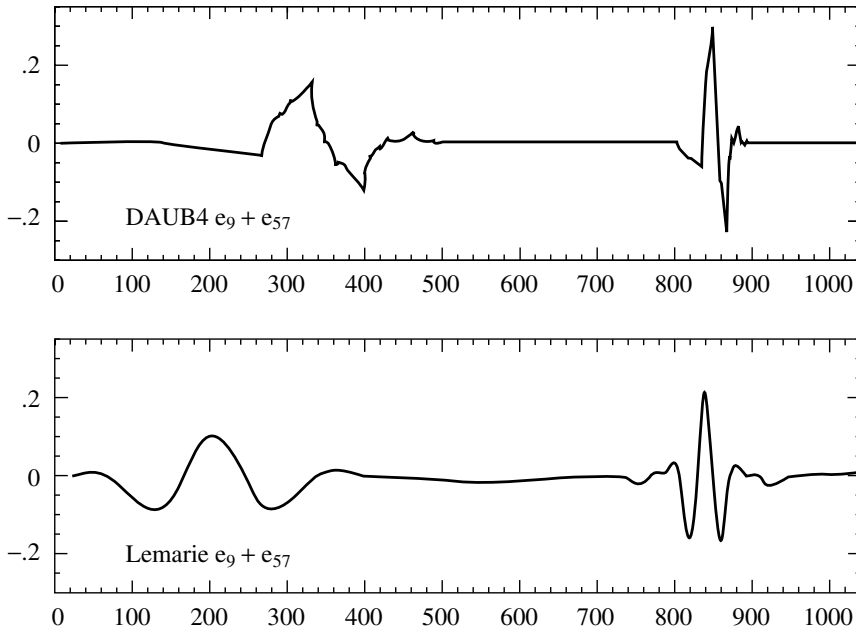


Figure 13.10.2. More wavelets, here generated from the sum of two unit vectors, $\mathbf{e}_9 + \mathbf{e}_{57}$, which are in different hierarchical levels of scale, and also at different spatial positions. DAUB4 wavelets (top) are defined by a filter in coordinate space (equation 13.10.5), while Lemarie wavelets (bottom) are defined by a filter most easily written in Fourier space (equation 13.10.14).

Note that the fact that wavelets are not smooth does not prevent their having exact representations for some smooth functions, as demanded by their approximation order p . The continuity of a wavelet is not the same as the continuity of functions that a set of wavelets can represent. For example, DAUB4 can represent (piecewise) linear functions of arbitrary slope: In the correct linear combinations, the cusps all cancel out. Every increase of two in the number of coefficients allows one higher order of polynomial to be exactly represented.

Figure 13.10.2 shows the result of performing the inverse DWT on the input vector $\mathbf{e}_9 + \mathbf{e}_{57}$, again for the two different particular wavelets. Since 9 lies early in the hierarchical range of 8–15, that wavelet lies on the left side of the picture. Since 57 lies in a later (smaller-scale) hierarchy, it is a narrower wavelet; in the range of 32–63 it is toward the end, so it lies on the right side of the picture. Note that smaller-scale wavelets are taller, so as to have the same squared integral.

13.10.4 Wavelet Filters in the Fourier Domain

The Fourier transform of a set of filter coefficients c_j is given by

$$H(\omega) = \sum_j c_j e^{ij\omega} \quad (13.10.8)$$

Here H is a function periodic in 2π , and it has the same meaning as before: It is the wavelet filter, now written in the Fourier domain. A very useful fact is that the orthogonality conditions for the c 's (e.g., equation 13.10.3 above) collapse to two

simple relations in the Fourier domain,

$$\frac{1}{2}|H(0)|^2 = 1 \quad (13.10.9)$$

and

$$\frac{1}{2}[|H(\omega)|^2 + |H(\omega + \pi)|^2] = 1 \quad (13.10.10)$$

Likewise, the approximation condition of order p (e.g., equation 13.10.4 above) has a simple formulation, requiring that $H(\omega)$ have a p th order zero at $\omega = \pi$, or (equivalently)

$$H^{(m)}(\pi) = 0 \quad m = 0, 1, \dots, p-1 \quad (13.10.11)$$

It is thus relatively straightforward to invent wavelet sets in the Fourier domain. You simply invent a function $H(\omega)$ satisfying equations (13.10.9) – (13.10.11). To find the actual c_j 's applicable to a data (or s -component) vector of length N , and with periodic wraparound as in matrices (13.10.1) and (13.10.2), you invert equation (13.10.8) by the discrete Fourier transform

$$c_j = \frac{1}{N} \sum_{k=0}^{N-1} H(2\pi \frac{k}{N}) e^{-2\pi i j k / N} \quad (13.10.12)$$

The quadrature mirror filter G (reversed c_j 's with alternating signs), incidentally, has the Fourier representation

$$G(\omega) = e^{-i\omega} H^*(\omega + \pi) \quad (13.10.13)$$

where the asterisk denotes complex conjugation.

In general, the above procedure will *not* produce wavelet filters with compact support. In other words, all N of the c_j 's, $j = 0, \dots, N-1$ will in general be nonzero (though they may be rapidly decreasing in magnitude). The Daubechies wavelets, or other wavelets with compact support, are specially chosen so that $H(\omega)$ is a trigonometric polynomial with only a small number of Fourier components, guaranteeing that there will be only a small number of nonzero c_j 's.

On the other hand, there is sometimes no particular reason to demand compact support. Giving it up in fact allows the ready construction of relatively smoother wavelets (higher values of p). Even without compact support, the convolutions implicit in the matrix (13.10.1) can be done efficiently by FFT methods.

Lemarie's wavelet (see [4]) has $p = 4$, does not have compact support, and is defined by the choice of $H(\omega)$,

$$H(\omega) = \left[2(1-u)^4 \frac{315 - 420u + 126u^2 - 4u^3}{315 - 420v + 126v^2 - 4v^3} \right]^{1/2} \quad (13.10.14)$$

where

$$u \equiv \sin^2 \frac{\omega}{2} \quad v \equiv \sin^2 \omega \quad (13.10.15)$$

It is beyond our scope to explain where equation (13.10.14) comes from. An informal description is that the quadrature mirror filter $G(\omega)$ deriving from equation (13.10.14) has the property that it gives identically zero when applied to any function whose odd-numbered samples are equal to the cubic spline interpolation of its even-numbered samples. Since this class of functions includes many very smooth members, it follows that $H(\omega)$ does a good job of truly selecting a function's smooth information content. Sample Lemarie wavelets are shown in Figure 13.10.2.

13.10.5 Daubechies Wavelets on the Interval

The discrete wavelet transforms that we have seen thus far are periodic and thus “live on a circle.”. Wavelets close to one edge of the data vector have tails that wrap around to the other edge. Said differently, some components of a discrete wavelet transform depend on data values at both ends of the data vector.

Most of the time, this periodicity is merely something between a curiosity and a minor nuisance, exactly like the discrete Fourier transform’s similar periodicity. Similar simple workarounds (e.g., zero-padding of the data) apply. Occasionally, however, the wraparound can produce undesirable effects, for example when the data differ by orders of magnitude at the two ends, or are smooth at one end but unsmooth at the other.

By modifying the coefficients of the wavelet filters near the two ends of the data vector, it is possible to produce wavelets that utilize only local data at each edge, that is, wavelets that “live on the interval” instead of on the circle. For such wavelets, the orthogonal matrix analogous to (13.10.1) is purely band-diagonal, and is identical to (13.10.1) except for modifications in the first and last few rows. Various such constructions have been proposed. Our favorite is that of [7].

One wrinkle needs to be mentioned: We would hope that those modified rows of the new matrix that are “detail filters” have the property of giving exactly zero when applied to smooth polynomial sequences like 1, 1, 1, 1, 1 or 1, 2, 3, 4, 5. Indeed, all the period wavelets previously discussed have this property. Alas, this condition, plus orthogonality, imposes too many constraints on the coefficients, and is unachievable. It turns out, however, that a simple linear preconditioning of the first and last few data points (that is, replacing the values by linear combinations of themselves) restores the desired property. The preconditioning is done only once in the transform, *not* at every pyramidal level. This need for preconditioning (with a corresponding postconditioning for the inverse) is the reason that our Wavelet abstract class has a method named `condition`. Finally we get to use it in a nontrivial way!

Here is an implementation of DAUB4 wavelets on the interval as a class derived from Wavelet, compatible for use in `wt1`. The ugliness of the code reflects only the large number of new coefficients that must be provided. If you want to implement higher DAUB n ’s on the interval, you’ll need even more coefficients, as found in [6] or [5].

```
struct Daub4i : Wavelet {
    void filt(VecDoub_IO &a, const Int n, const Int isign) {
        Applies the Cohen-Daubechies-Vial 4-coefficient wavelet on the interval filter to data vector
        a[0..n-1] (for isign=1) or applies its transpose (for isign=-1). Used hierarchically by
        routines wt1 and wtn.
        const Doub C0=0.4829629131445341, C1=0.8365163037378077,
            C2=0.2241438680420134, C3=-0.1294095225512603;
        const Doub R00=0.603332511928053, R01=0.690895531839104,
            R02=-0.398312997698228, R10=-0.796543516912183, R11=0.546392713959015,
            R12=-0.258792248333818, R20=0.0375174604524466, R21=0.457327659851769,
            R22=0.850088102549165, R23=0.223820356983114, R24=-0.129222743354319,
            R30=0.0100372245644139, R31=0.122351043116799, R32=0.227428111655837,
            R33=-0.836602921223654, R34=0.483012921773304, R43=0.443149049637559,
            R44=0.767556669298114, R45=0.374955331645687, R46=0.190151418429955,
            R47=-0.194233407427412, R53=0.231557595006790, R54=0.401069519430217,
            R55=-0.717579999353722, R56=-0.363906959570891, R57=0.371718966535296,
            R65=0.230389043796969, R66=0.434896997965703, R67=0.870508753349866,
            R75=-0.539822500731772, R76=0.801422961990337, R77=-0.257512919478482;
```

wavelet.h

```

Int nh,i,j;
if (n < 8) return;
VecDoub wksp(n);
nh = n >> 1;
if (isign >= 0) {
    wksp[0] = R00*a[0]+R01*a[1]+R02*a[2];
    wksp[nh] = R10*a[0]+R11*a[1]+R12*a[2];
    wksp[1] = R20*a[0]+R21*a[1]+R22*a[2]+R23*a[3]+R24*a[4];
    wksp[nh+1] = R30*a[0]+R31*a[1]+R32*a[2]+R33*a[3]+R34*a[4];
    for (i=2,j=3;j<n-4;j+=2,i++) {
        wksp[i] = C0*a[j]+C1*a[j+1]+C2*a[j+2]+C3*a[j+3];
        wksp[i+nh] = C3*a[j]-C2*a[j+1]+C1*a[j+2]-C0*a[j+3];
    }
    wksp[nh-2] = R43*a[n-5]+R44*a[n-4]+R45*a[n-3]+R46*a[n-2]+R47*a[n-1];
    wksp[n-2] = R53*a[n-5]+R54*a[n-4]+R55*a[n-3]+R56*a[n-2]+R57*a[n-1];
    wksp[nh-1] = R65*a[n-3]+R66*a[n-2]+R67*a[n-1];
    wksp[n-1] = R75*a[n-3]+R76*a[n-2]+R77*a[n-1];
} else {
    wksp[0] = R00*a[0]+R10*a[nh]+R20*a[1]+R30*a[nh+1];
    wksp[1] = R01*a[0]+R11*a[nh]+R21*a[1]+R31*a[nh+1];
    wksp[2] = R02*a[0]+R12*a[nh]+R22*a[1]+R32*a[nh+1];
    if (n == 8) {
        wksp[3] = R23*a[1]+R33*a[5]+R43*a[2]+R53*a[6];
        wksp[4] = R24*a[1]+R34*a[5]+R44*a[2]+R54*a[6];
    } else {
        wksp[3] = R23*a[1]+R33*a[nh+1]+C0*a[2]+C3*a[nh+2];
        wksp[4] = R24*a[1]+R34*a[nh+1]+C1*a[2]-C2*a[nh+2];
        wksp[n-5] = C2*a[nh-3]+C1*a[n-3]+R43*a[nh-2]+R53*a[n-2];
        wksp[n-4] = C3*a[nh-3]-C0*a[n-3]+R44*a[nh-2]+R54*a[n-2];
    }
    for (i=2,j=5;i<nh-3;i++) {
        wksp[j++] = C2*a[i]+C1*a[i+nh]+C0*a[i+1]+C3*a[i+nh+1];
        wksp[j++] = C3*a[i]-C0*a[i+nh]+C1*a[i+1]-C2*a[i+nh+1];
    }
    wksp[n-3] = R45*a[nh-2]+R55*a[n-2]+R65*a[nh-1]+R75*a[n-1];
    wksp[n-2] = R46*a[nh-2]+R56*a[n-2]+R66*a[nh-1]+R76*a[n-1];
    wksp[n-1] = R47*a[nh-2]+R57*a[n-2]+R67*a[nh-1]+R77*a[n-1];
}
for (i=0;i<n;i++) a[i]=wksp[i];
}
void condition(VecDoub_IO &a, const Int n, const Int isign) {
    Doub t0,t1,t2,t3;
    if (n < 4) return;
    if (isign >= 0) {
        t0 = 0.324894048898962*a[0]+0.0371580151158803*a[1];
        t1 = 1.00144540498130*a[1];
        t2 = 1.08984305289504*a[n-2];
        t3 = -0.800813234246437*a[n-2]+2.09629288435324*a[n-1];
        a[0]=t0; a[1]=t1; a[n-2]=t2; a[n-1]=t3;
    } else {
        t0 = 3.07792649138669*a[0]-0.114204567242137*a[1];
        t1 = 0.998556681198888*a[1];
        t2 = 0.917563310922261*a[n-2];
        t3 = 0.350522032550918*a[n-2]+0.477032578540915*a[n-1];
        a[0]=t0; a[1]=t1; a[n-2]=t2; a[n-1]=t3;
    }
}
};

```

Do you really need wavelets on the interval, instead of ordinary, periodic wavelets? Occasionally, yes. If you look ahead to Figure 13.10.6, which is a graphical display of two-dimensional wavelet coefficients, you can see the difference between

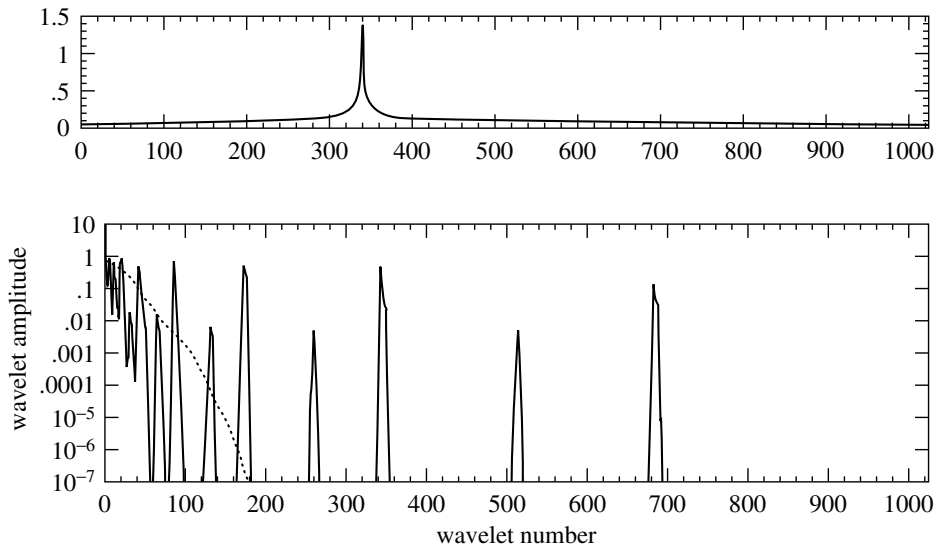


Figure 13.10.3. Top: Arbitrary test function, with cusp, sampled on a vector of length 1024. Bottom: Absolute value of the 1024 wavelet coefficients produced by the discrete wavelet transform of the function. Note log scale. The dotted curve plots the same amplitudes when sorted by decreasing size. One sees that only 130 out of 1024 coefficients are larger than 10^{-4} (or larger than about 10^{-5} times the largest coefficient, whose value is ~ 10).

allowing and suppressing wraparound.

13.10.6 Truncated Wavelet Approximations

Most of the usefulness of wavelets rests on the fact that wavelet transforms can usefully be severely truncated, that is, turned into sparse expansions. The case of Fourier transforms is different: FFTs are ordinarily used without truncation, to compute fast convolutions, for example. This works because the convolution operator is particularly simple in the Fourier basis. There are not, however, any standard mathematical operations that are especially simple in the wavelet basis.

To see how truncation works, consider the simple example shown in Figure 13.10.3. The upper panel shows an arbitrarily chosen test function, smooth except for a square-root cusp, sampled onto a vector of length 2^{10} . The bottom panel (solid curve) shows, on a log scale, the absolute value of the vector's components after it has been run through the DAUB4 discrete wavelet transform. One notes, from right to left, the different levels of hierarchy, 512–1023, 256–511, 128–255, etc. Within each level, the wavelet coefficients are nonnegligible only very near the location of the cusp, or very near the left and right boundaries of the hierarchical range (edge effects).

The dotted curve in the lower panel of Figure 13.10.3 plots the same amplitudes as the solid curve, but sorted into decreasing order of size. One can read off, for example, that the 130th largest wavelet coefficient has an amplitude less than 10^{-5} of the largest coefficient, whose magnitude is ~ 10 (power or square integral ratio less than 10^{-10}). Thus, the example function can be represented quite accurately by only 130, rather than 1024, coefficients — the remaining ones being set to zero.

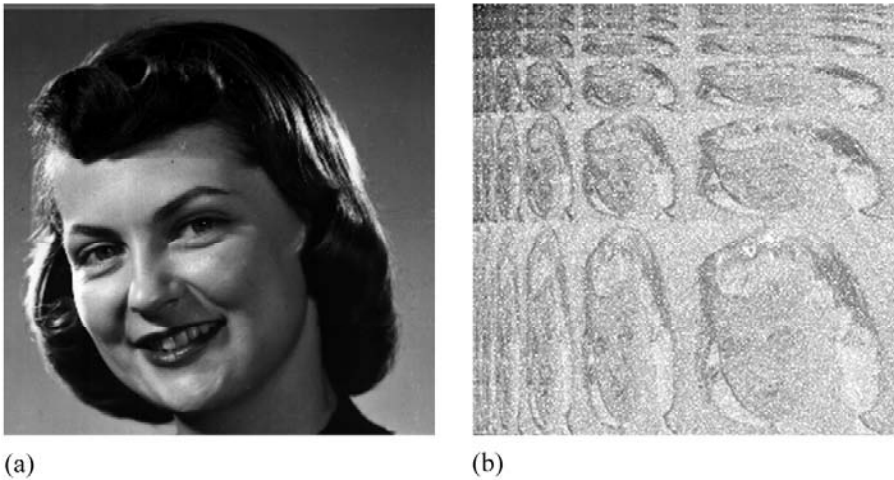


Figure 13.10.4. (a) Two-dimensional array of intensities (i.e., a photograph) and (b) its two-dimensional discrete wavelet transform. Darker pixels represent wavelet components that are larger in magnitude, on a logarithmic scale. Wavelets number from the upper-left corner, where the “smooth” information content is encoded.

```

        wlet.condition(wksp,n,1);
        for(nt=n;nt>=4;nt >>= 1) wlet.filt(wksp,nt,sign);
    } else {
        Or inverse transform.
        for(nt=4;nt<=n;nt <<= 1) wlet.filt(wksp,nt,sign);
        wlet.condition(wksp,n,-1);
    }
    for (i3=i1+i2,k=0;k<n;k++,i3+=nprev) a[i3]=wksp[k];
    Copy back from workspace.
}
    }
}
    nprev=nnew;
}

```

Here, as before, `wlet` is a `Wavelet` object that embodies a particular wavelet filter and (if required) pre-conditioner.

Figure 13.10.4 shows a sample image and its wavelet transform, represented graphically.

13.10.8 Compression of Images

An immediate application of the multidimensional transform `wtn` is to image compression. The overall procedure is to take the wavelet transform of a digitized image, and then to “allocate bits” among the wavelet coefficients in some highly nonuniform, optimized, manner. As already mentioned, large wavelet coefficients get quantized accurately, while small coefficients are quantized coarsely with only a bit or two — or else are truncated completely. If the resulting quantization levels are still statistically nonuniform, they may then be further compressed by a technique like Huffman coding (§22.5).

While a more detailed description of the “back end” of this process, namely the quantization and coding of the image, is beyond our scope, it is quite straightforward



Figure 13.10.5. (a) IEEE test image, 256×256 pixels with 8-bit grayscale. (b) The image is transformed into the wavelet basis; 77% of its wavelet components are set to zero (those of smallest magnitude); it is then reconstructed from the remaining 23%. (c) Same as (b), but 94.5% of the wavelet components are deleted. (d) Same as (c), but the Fourier transform is used instead of the wavelet transform. Wavelet coefficients are better than the Fourier coefficients at preserving relevant details.

to demonstrate the “front-end” wavelet encoding with a simple truncation: We keep (with full accuracy) all wavelet coefficients larger than some threshold, and we delete (set to zero) all smaller wavelet coefficients. We can then adjust the threshold to vary the fraction of preserved coefficients.

Figure 13.10.5 shows a sequence of images that differ in the number of wavelet coefficients that have been kept. The original picture (a), which is an official IEEE test image, has 256 by 256 pixels with an 8-bit grayscale. The two reproductions following are reconstructed with 23% (b) and 5.5% (c) of the 65536 wavelet coefficients. The latter image illustrates the kind of compromises made by the truncated wavelet representation. High-contrast edges (the model’s right cheek and hair highlights, e.g.) are maintained at a relatively high resolution, while low-contrast areas (the model’s left eye and cheek, e.g.) are washed out into what amounts to large constant pixels. Figure 13.10.5(d) is the result of performing the identical procedure with Fourier, instead of wavelet, transforms: The figure is reconstructed from the 5.5% of 65536 real Fourier components having the largest magnitudes. One sees that, since sines and cosines are nonlocal, the resolution is uniformly poor across the picture; also, the deletion of any components produces a mottled “ringing” everywhere. (Practical Fourier image compression schemes therefore break up an image

into small blocks of pixels, 16×16 , say, and do rather elaborate smoothing across block boundaries when the image is reconstructed.)

Viewers will sometimes choose (b) over (a), in Figure 13.10.5, as the superior image. The reason is that a “little bit” of wavelet compression has the effect of *denoising* the image. See [8] for a rigorous development.

13.10.9 Fast Solution of Linear Systems

There are interesting applications of wavelets to linear algebra. The basic idea [1] is to think of an integral operator (that is, a large matrix) as a digital image. Suppose that the operator compresses well under a two-dimensional wavelet transform, i.e., that a large fraction of its wavelet coefficients are so small as to be negligible. Then any linear system involving the operator becomes a sparse system in the wavelet basis. In other words, to solve

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \quad (13.10.17)$$

we first wavelet-transform the operator $\mathbf{A}$ and the right-hand side $\mathbf{b}$ by

$$\tilde{\mathbf{A}} \equiv \mathbf{W} \cdot \mathbf{A} \cdot \mathbf{W}^T, \quad \tilde{\mathbf{b}} \equiv \mathbf{W} \cdot \mathbf{b} \quad (13.10.18)$$

where $\mathbf{W}$ represents the one-dimensional wavelet transform, then solve

$$\tilde{\mathbf{A}} \cdot \tilde{\mathbf{x}} = \tilde{\mathbf{b}} \quad (13.10.19)$$

and finally transform to the answer by the inverse wavelet transform

$$\mathbf{x} = \mathbf{W}^T \cdot \tilde{\mathbf{x}} \quad (13.10.20)$$

(Note that the routine `wtn` does the complete transformation of $\mathbf{A}$ into $\tilde{\mathbf{A}}$.)

A typical integral operator that compresses well into wavelets has arbitrary (or even nearly singular) elements near its main diagonal, but becomes smooth away from the diagonal. An example might be

$$A_{ij} = \begin{cases} -1 & \text{if } i = j \\ |i - j|^{-1/2} & \text{otherwise} \end{cases} \quad (13.10.21)$$

Figure 13.10.6 shows a graphical representation of the wavelet transform of this matrix, where i and j range over $0 \dots 255$, using the DAUB4 wavelet, both in its conventional, periodic, implementation, and its modified form on the interval. Elements larger in magnitude than 10^{-3} times the maximum element are shown as black pixels, while elements between 10^{-3} and 10^{-6} are shown in gray. White pixels are $< 10^{-6}$. The indices i and j each number from the lower left.

In the figure, one sees the hierarchical decomposition into power-of-two sized blocks. At the edges or corners of the various blocks, one sees edge effects caused by the wraparound wavelet boundary conditions. Apart from edge effects, within each block, the nonnegligible elements are concentrated along the block diagonals. This is a statement that, for this type of linear operator, a wavelet is coupled mainly to near neighbors in its own hierarchy (square blocks along the main diagonal) and near neighbors in other hierarchies (rectangular blocks off the diagonal).

The number of nonnegligible elements in a matrix like that in Figure 13.10.6 scales only as N , the linear size of the matrix; as a rough rule of thumb it is about

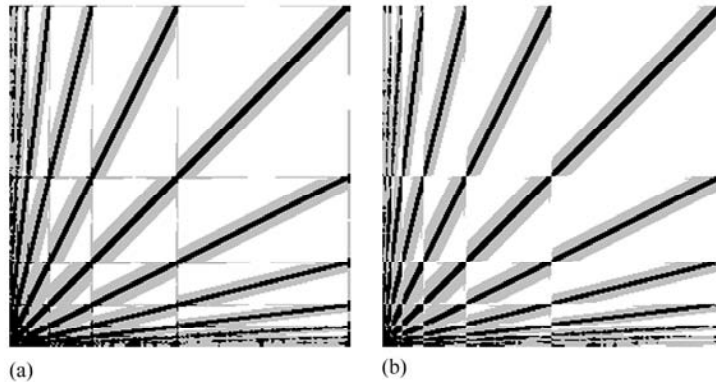


Figure 13.10.6. Wavelet transform of a 256×256 matrix, represented graphically. The original matrix has a discontinuous cusp along its diagonal, decaying smoothly away on both sides of the diagonal. In wavelet basis, the matrix becomes sparse: Components larger than 10^{-3} are shown as black, components larger than 10^{-6} as gray, and smaller-magnitude components are white. The matrix indices i and j number from the lower left. (a) Ordinary DAUB4 (periodic) is used. (b) Modified DAUB4 on the interval is used, eliminating wraparound artifacts and producing a more regular pattern of significant components.

$10N \log_{10}(1/\epsilon)$, where ϵ is the truncation level, e.g., 10^{-6} . For a 2000 by 2000 matrix, then, the matrix is sparse by a factor on the order of 30.

Various numerical schemes can be used to solve sparse linear systems of this “hierarchically band-diagonal” form. Beylkin, Coifman, and Rokhlin [1] make the interesting observations that (1) the product of two such matrices is itself hierarchically band-diagonal (truncating, of course, newly generated elements that are smaller than the predetermined threshold ϵ); and, moreover, that (2) the product can be formed in order N operations.

Fast matrix multiplication enables finding the matrix inverse by Schultz’s (or Hotelling’s) method; see §2.5.

Other schemes are also possible for fast solution of hierarchically band-diagonal forms. For example, one can use the conjugate gradient method, implemented in §2.7 as `linbcg`.

CITED REFERENCES AND FURTHER READING:

- Daubechies, I. 1992, *Wavelets* (Philadelphia: S.I.A.M.).
- Strang, G. 1989, “Wavelets and Dilation Equations: A Brief Introduction,” *SIAM Review*, vol. 31, pp. 614–627.
- Beylkin, G., Coifman, R., and Rokhlin, V. 1991, “Fast Wavelet Transforms and Numerical Algorithms,” *Communications on Pure and Applied Mathematics*, vol. 44, pp. 141–183.[1]
- Daubechies, I. 1988, “Orthonormal Bases of Compactly Supported Wavelets,” *Communications on Pure and Applied Mathematics*, vol. 41, pp. 909–996.[2]
- Vaidyanathan, P.P. 1990, “Multirate Digital Filters, Filter Banks, Polyphase Networks, and Applications,” *Proceedings of the IEEE*, vol. 78, pp. 56–93.[3]
- Mallat, S.G. 1989, “A Theory for Multiresolution Signal Decomposition: The Wavelet Representation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 11, pp. 674–693.[4]
- Cohen, A. 1993, “Tables for Wavelet Filters Adapted to Life on an Interval,” multiple Web sites; mirrored at <http://www.nr.com/contrib>. [5]